

BENER PERTAMA

Muhammad Hanif galuh S

51422069

3ia01

1. . Jelaskan mengapa pentingnya sebuah program terintegrasi dengan database ! (5 point)

Jawab:

Adanya penyimpanan terpusat dimana ada satu wadah, efisiensi operasi, keamanan data, kemudahan skalabilitas

2. . Dalam konteks pengembangan REST API dengan Golang, jelaskan mengapa pemilihan struktur project (project structure) yang tepat itu penting dan sebutkan minimal 3 folder/package yang umumnya ada dalam struktur project Golang! (5 point)

Jawab:

Memudahkan perbaikan, memudahkan testing, mempermudah kolaborasi

Models/: berisi definisi struktur dan method

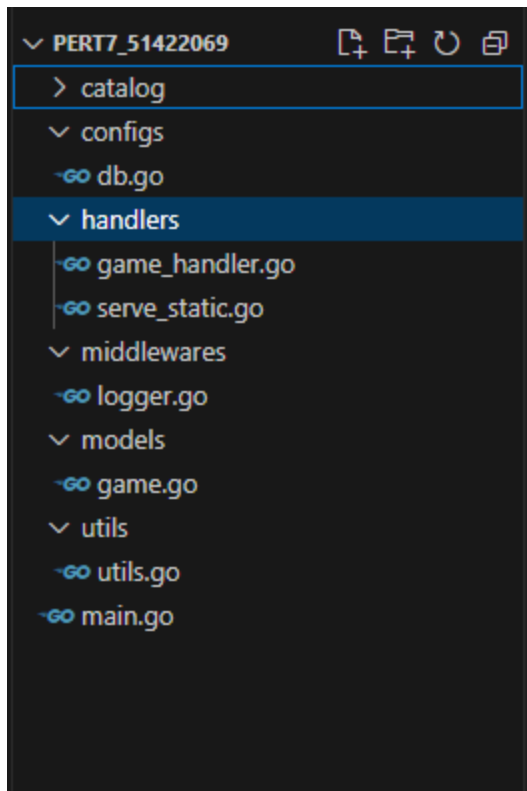
Handlers/: berisi logic untuk routing dan endpoint

Middlewares/: isinya ini auth, dan log

3.

```
mysql> use steam_catalog;
mysql> SELECT * FROM games;
+----+-----+-----+-----+-----+
| id_game | nama_game | developer | genre | harga |
+----+-----+-----+-----+-----+
| 1 | Cyber Battle | Cyber Studios | Action | 49.99 |
| 2 | Fantasy Quest | DreamWorks Games | RPG | 59.99 |
| 3 | Speed Racer | Fastline Devs | Racing | 29.99 |
+----+-----+-----+-----+-----+
3 rows in set (0.001 sec)
```

1. Buatlah struktur direktori proyek seperti dibawah ini ! (5 point)



2. Pada file main.go, import library yang diperlukan dan buatlah function main() yang akan menginisialisasi dan menjalankan server pada port xxxx (empat digit terakhir npm). Fungsi ini harus dapat melakukan koneksi ke database, menangani kemungkinan kegagalan koneksi ke database, menyiapkan file server untuk konten statis dari folder 'catalog', serta mendaftarkan route yang diperlukan untuk API games. Pastikan juga untuk mengimplementasikan middleware logging untuk mencatat semua request yang masuk ke server. Perhatikan bahwa koneksi database harus ditutup dengan benar ketika server berhenti beroperasi ! (10 point).

```
2. package main

import (
    "fmt"
    "log"
    "net/http"
    "pert7_51422069/configs"
    "pert7_51422069/handlers"
    "pert7_51422069/middlewares"
    _ "github.com/go-sql-driver/mysql"
)

//No 2
func main() {
```

```

PORT := 8081

configs.ConnectDB()
if configs.DB == nil {
    log.Fatal("Database connection failed")
}
defer func() {
    if err := configs.DB.Close(); err != nil {
        log.Printf("Error closing database connection: %v", err)
    }
}()

mux := http.NewServeMux()
fileServer := http.FileServer(http.Dir("catalog"))
mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
    handlers.ServeStaticFile(w, r, "catalog", fileServer)
})
mux.HandleFunc("/api/games/", handlers.HandleGames)
mux.HandleFunc("/api/games", handlers.HandleGames)

// Apply middleware
loggedMux := middlewares.LogRequestHandler(mux)

fmt.Printf("Server berjalan di http://localhost:%d\n", PORT)
log.Fatal(http.ListenAndServe(fmt.Sprintf(":%d", PORT), loggedMux))
}

```

3. Pada file configs/db.go, import library yang diperlukan dan buat fungsi ConnectDB() yang akan menginisialisasi koneksi database (10 point)

```

3. package configs

import (
    "database/sql"
    "log"
    "time"

    _ "github.com/go-sql-driver/mysql"
)

var DB *sql.DB

//No 3

```

```

func ConnectDB(){
    var err error
    db, ERR = sql.Open("mysql", "root@tcp(127.0.0.1:3306)/steam_catalog")
    if err != nil{
        log.Fatalf("failed to open database connection:", err)
    }
    DB.SetMaxOpenConns(10)
    DB.SetMaxIdleConns(5)
    DB.SetConnMaxLifetime(time.Minute * 5)

    err = DB.Ping()
    if err != nil{
        log.Fatalf("failed to ping database:", err)
    }

    _, err = DB.Query("SELECT 1 FROM gamees LIMIT 1")
    if err != nil{
        LOG.FATAL("FAILED TO QUERY GAMES TABLE:", err)
    }

    log.Println("successfull connected to database")
}
//isi lah potongan kode yang hilang dibawah ini

```

4. Pada file models/game.go, buatlah struct Game dengan lima field: ID, Nama, Developer, Genre, dan Harga (5 point)

```

package models

type Game struct{
    ID int `json:"id_game"`
    Nama string `json:"nama_game"`
    Developer string `json:"developer"`
    Genre string `json:"genre"`
    Harga float64 `json:"harga"`
}

```

5. Pada file utils/utils.go, buatlah fungsi RespondJSON untuk mengirim response dalam format JSON. Fungsi ini menerima http.ResponseWriter dan data interface{} sebagai parameter (5 point).

```
//no 5
package utils

import (
    "encoding/json"
    "net/http"
)

func RespondJSON(w http.ResponseWriter, data interface{}) {
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(data)
}
```

6. Pada file handlers/serve_static.go, buatlah fungsi ServeStaticFile untuk melayani file statis dari folder catalog. Fungsi ini harus dapat menangani path URL, melakukan pengecekan file, dan mengarahkan kembali ke halaman utama jika file tidak ditemukan. Pastikan request ke path '/api/' tidak dilayani oleh file server (10 point).

```
//no 6
package handlers

import (
    "net/http"
    "os"
    "path/filepath"
    "strings"
)

func ServeStaticFile(w http.ResponseWriter, r *http.Request, baseDir string,
fileServer http.Handler) {
    path := strings.TrimPrefix(r.URL.Path, "/")
    fullPath := filepath.Join(baseDir, path)

    if strings.HasPrefix(r.URL.Path, "/api/") {
        http.NotFound(w, r)
        return
    }

    _, err := os.Stat(fullPath)
    if err != nil {
        http.Redirect(w, r, "/", http.StatusTemporaryRedirect)
        return
    }
}
```

```

    }
    fileServer.ServeHTTP(w, r)
}

```

7. Pada file `handlers/game_handler.go`, buatlah API endpoint untuk operasi CRUD pada data game. Buatlah fungsi utama `HandleGames` yang akan mengarahkan request ke fungsi yang sesuai berdasarkan method HTTP. Implementasikan juga fungsi `handleGetGames` untuk mengambil data game (baik semua game maupun game spesifik), `handlePostGame` untuk menambah game baru, `handleUpdateGame` untuk memperbarui data game, dan `handleDeleteGame` untuk menghapus game. Pastikan setiap fungsi menangani error dengan tepat dan memberikan response yang sesuai (40 point).

```

//No7
package handlers

import (
    "database/sql"
    "encoding/json"
    "net/http"
    "pert7_51422069/configs"
    "pert7_51422069/models"
    "pert7_51422069/utils"
    "strconv"
)

func HandleGames(w http.ResponseWriter, r *http.Request) {
    path := r.URL.Path
    var id int
    var err error

    if len(path) > len("/api/games/") {
        id, err = strconv.Atoi(path[len("/api/games/"):])
        if err != nil {
            http.Error(w, "ID tidak valid", http.StatusBadRequest)
            return
        }
    } else {
        id = 0
    }

    switch r.Method {
    case http.MethodGet:
        handleGetGames(w, id)
    case http.MethodPost:
        handlePostGame(w, r)
    }
}

```

```

    case http.MethodPut:
        handleUpdateGame(w, r, id)
    case http.MethodDelete:
        handleDeleteGame(w, id)
    }
}

func handleGetGames(w http.ResponseWriter, id int) {
    if id == 0 {
        rows, err := configs.DB.Query("SELECT id_game, nama_game, developer,
genre, harga FROM games")
        if err != nil {
            http.Error(w, "Gagal mengambil data game",
http.StatusInternalServerError)
            return
        }
        defer rows.Close()

        var games []models.Game
        for rows.Next() {
            var game models.Game
            err := rows.Scan(&game.ID, &game>Nama, &game.Developer, &game.Genre,
&game.Harga)
            if err != nil {
                http.Error(w, "Gagal memproses data game",
http.StatusInternalServerError)
                return
            }
            games = append(games, game)
        }
        utils.RespondJSON(w, games)
    } else {
        var game models.Game
        err := configs.DB.QueryRow("SELECT id_game, nama_game, developer, genre,
harga FROM games WHERE id_game = ?", id).
            Scan(&game.ID, &game>Nama, &game.Developer, &game.Genre, &game.Harga)
        if err != nil {
            if err == sql.ErrNoRows {
                http.Error(w, "Game tidak ditemukan", http.StatusNotFound)
            } else {
                http.Error(w, "Gagal mengambil data game",
http.StatusInternalServerError)
            }
            return
        }
    }
}

```

```

        utils.RespondJSON(w, game)
    }
}

func handlePostGame(w http.ResponseWriter, r *http.Request) {
    var game models.Game
    if err := json.NewDecoder(r.Body).Decode(&game); err != nil {
        utils.RespondJSON(w, map[string]string{"error": "Data tidak valid"})
        return
    }

    _, err := configs.DB.Exec(
        "INSERT INTO games (nama_game, developer, genre, harga) VALUES (?, ?, ?, ?)",
        game>Nama, game.Developer, game.Genre, game.Harga)

    if err != nil {
        // This will be caught by your LogRequestHandler middleware
        utils.RespondJSON(w, map[string]string{"error": err.Error()})
        return
    }

    utils.RespondJSON(w, map[string]string{"message": "Game berhasil
ditambahkan!"})
}

func handleUpdateGame(w http.ResponseWriter, r *http.Request, id int) {
    var game models.Game
    if err := json.NewDecoder(r.Body).Decode(&game); err != nil {
        http.Error(w, "Data tidak valid", http.StatusBadRequest)
        return
    }

    result, err := configs.DB.Exec("UPDATE games SET nama_game=?, developer=?,
genre=?, harga=? WHERE id_game=?",
        game>Nama, game.Developer, game.Genre, game.Harga, id)
    if err != nil {
        http.Error(w, "Gagal memperbarui game", http.StatusInternalServerError)
        return
    }

    rowsAffected, _ := result.RowsAffected()
    if rowsAffected == 0 {
        http.Error(w, "Game dengan ID tersebut tidak ditemukan",
http.StatusNotFound)
        return
    }
}

```



```

    utils.RespondJSON(w, map[string]string{"message": "Game berhasil
diperbarui!"})
}

func handleDeleteGame(w http.ResponseWriter, id int) {
    result, err := configs.DB.Exec("DELETE FROM games WHERE id_game=?", id)
    if err != nil {
        http.Error(w, "Gagal menghapus game", http.StatusInternalServerError)
        return
    }

    rowsAffected, _ := result.RowsAffected()
    if rowsAffected == 0 {
        http.Error(w, "Game dengan ID tersebut tidak ditemukan",
http.StatusNotFound)
        return
    }

    utils.RespondJSON(w, map[string]string{"message": "Game berhasil dihapus!"})
}

```

8. Pada file middlewares/logger.go, buatlah fungsi LogRequestHandler untuk mencatat setiap request yang masuk ke server. Fungsi ini harus menerima dan meneruskan request ke handler berikutnya sambil melakukan logging method dan path URL request (10 point).

JAWAB:

```

//no 8
package middlewares

import (
    "log"
    "net/http"
)

func LogRequestHandler(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        log.Printf("Incoming request: %s %s", r.Method, r.URL.Path)
        next.ServeHTTP(w, r)
    })
}

```

